

在记忆中思考：记忆中的思考：回忆与后思让具有长期记忆的 LLM 成为可能

刘磊* liulei1497@gmail.com

香港中文大学深圳分校, 蚂蚁小组

胡彬彬, 张志强

{bin.hbb, lingyao.zzq}@antfin.com 蚂蚁金

服集团

杨晓燕

joyce.yxy@antgroup.com 蚂

蚁金服集团

顾金杰

jinjie.gjj@antfin.com 蚂

蚁金服集团

沈悦

[†]zhanying@antgroup.com

蚂蚁金服集团

张冠楠

zgn138592@antfin.com 蚂

蚁金服集团

摘要

记忆增强型大语言模型 (LLM) 在长期人机互动中表现出色, 它基本上依靠对历史的反复回忆和推理来生成高质量的回答。然而, 这种重复的回忆-推理步骤很容易产生思维偏差, 即在针对不同的问题回忆相同的历史记录时, 推理结果不一致。相反, 人类可以将想法保存在记忆中, 并在不重复推理的情况下进行回忆。受人类这种能力的启发, 我们提出了一种名为 TiM (Think-in-Memory) 的新型记忆机制, 它能让 LLM 保持一种进化的记忆, 用于存储会话流中的历史想法。TiM 框架包括两个关键阶段: (1) 在做出回应之前, LLM 代理从内存中回忆相关想法; (2) 在做出回应之后, LLM 代理进行后思考, 并将历史想法和新想法纳入内存以更新内存。因此, TiM 可以将思考后的想法保存为历史, 从而消除重复推理的问题。在此基础上, 我们制定了基本原则, 以便根据成熟的操作 (即插入、遗忘和合并操作) 在内存中组织思想, 从而实现思想的动态更新和演化。此外, 我们还在 TiM 中引入了位置敏感散列 (Locality-Sensitive Hashing), 以实现对长期对话的高效检索。我们在真实世界和模拟对话中进行了定性和定量实验, 涵盖了广泛的主题, 结果表明, 将现有的 LLM 与 TiM 结合使用, 可以显著提高它们在生成长期互动回复方面的性能。

关键词

大语言模型 反应生成 长期记忆

1 引言

大型语言模型 (LLM) 的巨大进步彻底改变了人类与人工智能 (AI) 系统之间的交互。正如 ChatGPT [1] 和 GPT-4 [2] 所展示的那样, 这些进步尤其体现在人机对话中的卓越性能。从金融[3]和医疗保健[4]到商业和客户服务[5], 这些先进的 LLM 在理解问题和生成相应回复方面表现出了非凡的能力。值得注意的是, 大型模型的规模、

这使得 LLMs 具备了类似人类的能力[6]。

尽管在大型语料库中预先训练过的 LLM 具有非凡的能力, 但基于 LLM 的人工智能代理在长期应用场景中仍面临一个重大限制, 即无法处理异常冗长的输入[7]。这在某些特定任务中尤为重要, 例如, 医疗人工智能助手[4]依赖于过去对话的症状来提供准确的临床诊断。因此, 不具备处理长期输入能力的 LLM 可能会因为遗忘重要的疾病症状而影响诊断的准确性 (见第 4.4)。因此, 有必要开发具有长期能力的人工智能系统, 以实现更准确、更可靠的交互。为了提高 LLM 处理长期输入的能力, 已经开展了多项研究。总的来说, 这些研究大致可分为两类: (1) **基于内部记忆的方法**[8], 旨在降低自我注意的计算成本, 以扩展序列长度。为了适应较长的输入文本, 应利用特殊的位置编码来学习相对位置。例如, [9] 探索了一种具有全局编码器标记的块本地变换器, 并结合额外的长输入预训练。(2) **基于外部记忆的方法** (也称为长期记忆机制 [10]) 一般利用物理空间作为记忆缓存来存储历史信息, 相关的历史信息可以从记忆缓存中读取, 从而在不遗忘的情况下增强 LLM。特别是, 令牌和

原始文本可以作为历史保留在内存中。例如, [11] 的研究表明, 在 BERT 嵌入的辅助下, 通过使用包含数万亿标记的外部内存缓存来增强 LLM, 可以显著提高性能[12]。需要注意的是, 基于令牌的内存机制需要调整 LLM 的架构来适应, 很难与不同的 LLM 结合使用。通过访问外部内存缓存, 增强型 LLM 在各种语言建模基准测试中取得了新的先进记录, 其性能普遍优于基于内部内存的方法。因此, 这项工作的重点是开发一种与 LLM 无关的外部内存机制, 以增强 LLM 的记忆能力。

一般来说, 内存增强型 LLM 的效用主要取决于其对外部内存缓存中的历史记录进行迭代调用和重复推理的能力。具体来说, 为了实现 n -th turn 之后的对话, LLM 需要重新理解和推理从第 0 次到 $(n-1)$ -th 对话的历史。例如, 如图 1, LLMs 所示要回答第 2 次和第 3 次对话的问题需要回忆从第 0 次到 $(n-1)$ -th 对话的历史。

和第 3 次对话的问题, LLMs 需要回忆第 1 次对话并进行两次推理。

*这项工作是在刘磊在蚂蚁金服实习时完成的。
通讯作者。

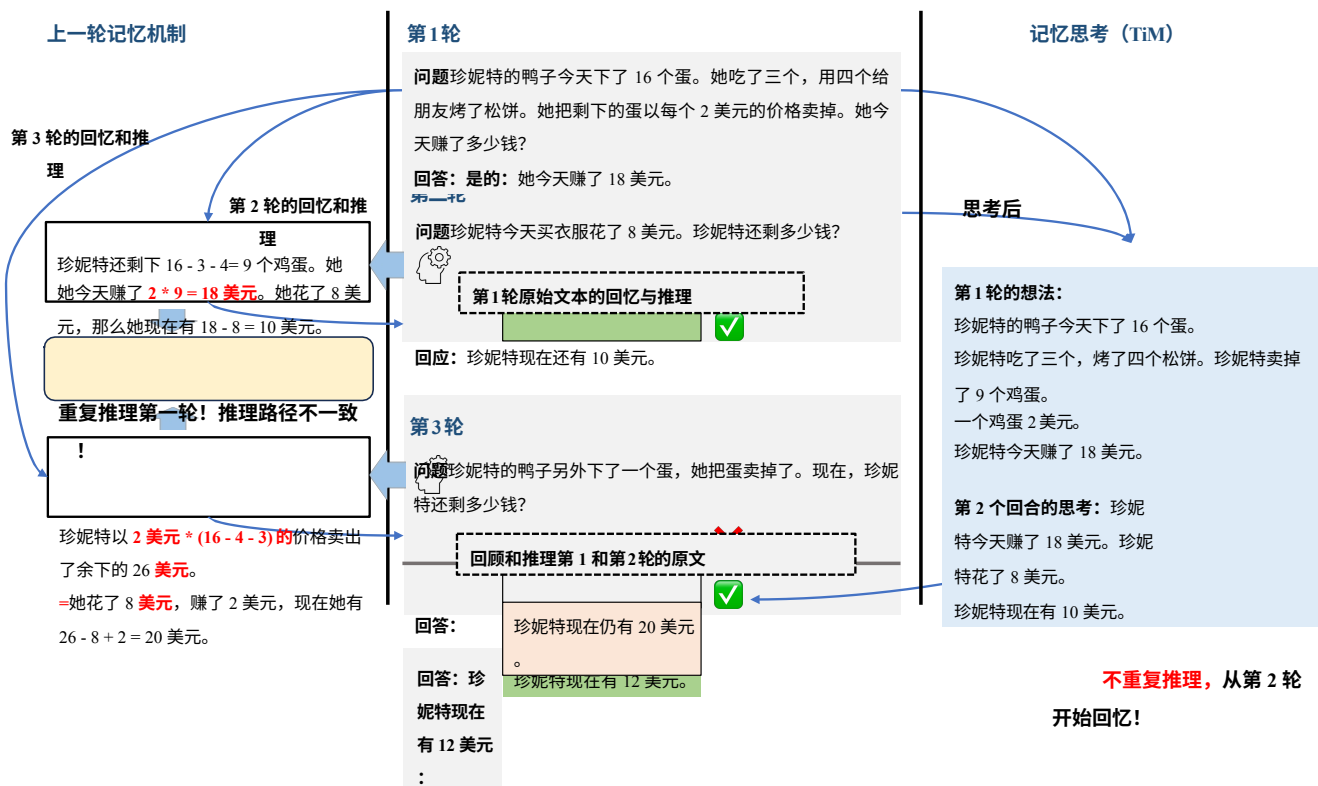


图 1：以前的记忆机制与我们提出的 TiM 的比较。(左图)：现有的记忆机制主要保存的是以前的原始文本，这就需要对相同的历史进行重复推理。这很容易导致不一致的推理路径（即左图红色部分）和错误的响应。(右图)：拟议的 TiM 存储了 LLM 在之前各轮中的想法，可以避免这种不一致，而无需重复推理（即右侧红色部分）。

遗憾的是，这种模式容易遇到一些问题，并有可能在实际应用中造成性能瓶颈。主要问题如下：

- **推理路径不一致。**先前的研究[13, 14]表明，LLMs 很容易为同一查询生成不同的推理路径。
推理路径。如图 1（左）所示，由于对上下文的推理不一致，LLM 会给出错误的响应。
- **令人不满意的检索成本。**为了检索相关的历史记录，前这些记忆机制需要计算问题与每段历史对话之间的成对相似性。
这对于长期对话来说非常耗时。

为了解决这些问题，我们希望通过类典型的元认知过程（metacognition）[15]，即大脑将想法保存为记忆而不是原始事件的细节，在记忆增强型 LLM 方面更进一步。因此，在这项工作中，我们提出了一个“记忆中思考”（Think-in-Memory, TiM）框架来模拟类似人类的记忆机制，从而使 LLMs 能够在长期交互场景中记住并有选择性地回忆起历史思想。具体来说，如图 2 所示，TiM 框架分为两个阶段：（1）在回忆阶段，LLM 通过回忆记忆中的相关想法来生成对新查询的回应；（2）在后思考阶段，LLM 对回应进行推理和思考，并将新想法保存到外部记忆中。此外，为了反映人类的认知过程，我们还制定了一些基本原则，以便根据成熟的操作（如插入、遗忘和合并操作）来组织内存中的想法，从而实现想法的动态更新和演变。具体来说，TiM 建立在以下基础之上

基于散列的检索机制（即位置敏感散列[16]），支持高效的“手入”（即插入想法）和“手出”（即调用想法）操作。此外，TiM 的设计与 LLM 无关，这意味着它可以与各种类型的语言模型相结合。这包括 ChatGPT [1] 等，闭源 LLM 以及 ChatGLM[17] 等开源 LLM。

这项工作的主要贡献概述如下：

- 我们提出了一种新颖的类人长期记忆机制--TiM，使 LLM 能够记忆并选择性地回忆想法。
回忆思想。TiM 可以让 LLM 在记忆中思考，而无需对长期历史进行重复推理。
- 我们根据已被证实的事实，制定了一些基本原则来组织记忆中的思想。我们基于成熟的操作方法，制定了一些基本原则来组织记忆中的思想。这反映了人类的认知过程，使记忆中的思想能够动态更新和演化。此外，我们还引入了一种基于哈希值的检索机制，以有效利用 TiM。
- 我们在多轮对话中进行了广泛的实验。
数据集。结果表明，我们的方法可以从以下几个方面显著提高 LLM 的性能（1）支持从开放领域到特定领域的各种主题；（2）支持中英文双语；（3）提高了回答的正确性和连贯性。

对话回合。建议的 TiM 允许代理处理长期对话，并在与用户多次对话后保留有用的历史信息。

长期对话，并在与用户多次对话后保留有用的历史信息。

3.1.1 主要组件。如图 2，所示我们的 TiM 由以下部分组成，共同为长期对话提供更准确、更连贯的回应：

- 代理 A：A 是一个预先训练好的 LLM 模型，用于促进动态对话，如 ChatGPT [1] 和 ChatGLM [17]。
- 内存缓存 M：M 是一个不断增长的键值对散列表，其中键是散列索引，值是单个思想。有关 M 的更多详情，请参阅第 3.2。节明确地说，M 支持表 1。所示的各种操作
- 基于散列的映射 $F(\cdot)$ ：对位置敏感的散列映射
哈希映射 $F(\cdot)$ ：引入了对位置敏感的哈希算法，以快速保存和查找 M 中的相关想法。

3.1.2 工作流程。整体框架分为两个阶段：

- **阶段-1：回忆和生成。**当用户提出一个新问题时，LLM 代理 A 会检索相关想法，以生成准确的回复。由于我们保存了自我生成的推理
这一阶段可以直接回忆并回答问题，而无需对原始历史对话文本进行反复推敲。
- **第二阶段：后思考与更新。**回答完问题后，我们让 LLM 代理对 QR 进行后思考，并将新生成的推理思路插入内存缓存 M。

3.2 内存缓存的存储

3.2.1 基于思维的系统。TiM 的存储系统 M 旨在通过自我生成的推理思想（定义 3）来保存人工智能与用户交互的知识。

定义 3.1)。每个想法 T 都以元组 (H_{idx}, T) 的格式存储，其中 H_{idx} 是哈希函数 $F(T)$ 获得的哈希索引。这

基于哈希值的存储不仅有助于快速检索记忆，而且还能促进记忆更新，提供历史思想的详细索引。

定义 3.1.1 归纳思想。归纳思想被定义为包含两个实体之间关系的文本，即满足关系三重 $(E_{\text{真实性}}, r_i, E_i)$ 。 $E_{\text{籍}}$ 是头实体
通过关系 r_i 与尾部实体 E_i 相连，其中 $i \in [0, N]$ 。
和 N 是关系编号。从概念上讲， $R_{\text{嗣}} = \{r_1, \dots, r_N\}$
由实体 $E_{\text{籍}}$ 的所有单跳关系组成。

将归纳思想用于 LLM 的主要挑战在于获得与关系三匹配的高质量句子。在此，我们提供两种获得归纳思想的解决方案：（1）用于开放信息提取的预训练模型，如 OpenIE [34]；（2）基于 LLM 的语境学习（In-context learning）。在这项工作中，我们采用了第二种解决方案，即利用 LLM Agent A 生成归纳式思维，如图 3。所示

3.2.2 基于哈希值的存储。我们的目标是按照一定的规则将归纳式想法保存到内存中，也就是说，为了提高效率，类似的想法应该保存在内存的同一组中。为此，我们采用哈希表作为 TiM 存储系统的架构，将相似的想法分配给相同的哈希索引。

给定一个查询，我们建议在多维嵌入空间中快速搜索其最近的想法，这可以通过位置敏感散列（LSH）方法来解决。散列方案

生成想法的提示

给定以下问题和回答对，请提取带有相应文本的关系（主题、关系、对象）：

例 1.输入

问题你有什么公司推荐给我吗？回答：我推荐 Google：我推荐 Google。

输出：

(公司, 推荐, 谷歌)。推荐的公司是 Google。

示例 2.输入

问题中国的首都是哪个城市？回答：北京：北京。

输出：

(中国、首都、北京)。

中国的首都是北京。

输入：

问题：你有什么书推荐给我吗？回答：我推荐《小王子》：我推荐《小王子》。

输出：

图 3：引发思考的提示示例。

LSH 的目的是为每个 d -dimension embedding vector $x \in \mathbb{R}^d$ 分配一个哈希索引 $F(x)$ ，其中附近的向量以更高的概率获得相同的哈希索引。我们利用随机投影来实现：

$$F(x) = \arg \max (|xR - xR|), \quad (1)$$

其中， R 是大小为 $(d, b/2)$ 的随机矩阵， b 是内存中的分组数。 $[;uv]$ 表示两个向量的连接。这种 LSH 方法是一种众所周知的 LSH 方案 [16]，其特点是易于实现。图 2 展示了 TiM 基于 LSH 的存储系统示意图。

3.3 记忆检索

在记忆存储的基础上，记忆检索对最相关的想法进行两阶段检索任务，即基于 LSH 的检索和基于相似性的检索。该范式的具体要点如下。

- **阶段-1：基于 LSH 的检索。**对于一个新的查询 Q ，我们首先根据 LLM 代理获取其嵌入向量 x 。然后，LSH 函数（即公式 1）可以生成查询的哈希索引。

根据 LSH 的特性，该散列值索引还能指出内存缓存中与其最接近的相似思想组。

- **阶段-2：基于相似性的检索。**在最近组内我们会计算查询与组中每个想法之间的成对相似度。然后， k 最重要的思想将作为相关历史记录被调用，以准确回答查询。它

给定下列想法，请删除反事实想法或相互矛盾的想法：

例 1.输入：

中国的首都是北京。中国的首都是上海。

美国的首都是华盛顿。美国首都是纽约。

输出：

中国的首都是北京。

美国的首都是华盛顿。

例 2.输入：

迈克尔喜欢踢足球。迈克尔不喜欢踢足球。詹姆斯喜欢游泳。

玛丽喜欢看书。

输出：詹姆斯喜欢游泳：

詹姆斯喜欢游泳。玛丽喜欢看书。

输入：[一组想法

[一组想法]

输出：詹姆斯喜欢游泳

遗忘想法提示

图 4：遗忘想法的提示示例。

应该注意的是，成对相似度是在一个组内计算的，而不是在整个记忆缓存中计算的，这比以前的记忆机制能实现更高效的检索。

3.4 记忆更新的组织

有了以上讨论的记忆存储和检索，LLM 的长时记忆能力就能得到很好的提升。受人类记忆的启发，需要一些基于成熟操作的组织原则来实现思想的动态更新和演化，如插入新思想、遗忘不重要思想、合并重复思想等，从而使记忆机制更加自然和适用。

从内存缓存的存储架构开始，TiM 采用哈希表存储自生成的想法，每个哈希索引对应一个包含相似想法的组。在同一组中，TiM 支持以下操作来组织内存中的想法：

- **插入**，即把新想法存储到内存中。生成想法的提示如图 3。所示
- **遗忘**，即从内存中删除不必要的想法，如相互矛盾的想法。该操作的提示符为如图 4。所示
- **合并**，即合并记忆中相似的想法，如具有相同头部实体的想法。该操作的提示如图 5 所示。

给定下列想法，请将相似的想法与相同的实体合并：

例 1.输入：

约翰是一名演员。约翰是一名导演。约翰是一名作家。迈克是一名教师。

输出：

约翰担任演员、导演和编剧。迈克是一名教师。

示例 2.输入

迈克尔喜欢踢足球。迈克尔喜欢打篮球。詹姆斯喜欢游泳。

玛丽喜欢看书。

输出：

迈克尔喜欢踢足球和打篮球。詹姆斯喜欢游泳。

玛丽喜欢看书。

输入：[一组想法

[一组想法]

输出：

合并想法提示

图 5：合并想法的提示示例。

表 1：以前的记忆机制与我们的记忆机制的组织结构比较。KG 表示知识图谱，Q-R 表示问题和回答对。

方法	内容	与 LLM 无关	插入	遗忘	合并
单片机 [33]	Q-R	✓	✓	■	■
RelationLM [7]	KG	■	✓	■	■
LongMem [10]	代币	■	✓	■	■
内存库 [32]	Q-R	✓	✓	✓	■
我们的 (TiM)	思考	✓	✓	✓	✓

3.5 参数高效调整

在计算资源有限的情况下，我们采用了一种名为低等级适应（Low- Rank Adaptation，LoRA）[27] 的计算效率高的微调方法。LoRA [27] 在保持原有权重不变的情况下优化秩分解矩阵对，从而有效减少可训练参数的数量。具体来说，考虑到线性层的定义为 $y = Wx$ ，LoRA 会根据以下条件对其进行微调 $y = Wx + BAx$ ，其中 $W \in \mathbb{R}^{d \times k}$ 、 $B \in \mathbb{R}^{d \times r}$ 、 $A \in \mathbb{R}^{r \times k}$ 、和 $r \ll \min(d, k)$ 。从根本上说，这一微调阶段可以使 LLM 适应多轮对话，为用户提供适当而有效的响应。

有效地响应用户。在所有实验中，我们将 LoRA rank 设置为 r 为 16，并对 LLM 模型进行 10 次历时训练。

3.6 有启发性的讨论

我们在表 1 中总结了以往的记忆机制和我们的方法，包括记忆内容、LLM 无关性和组织操作。从表 1：中可以观察到几个重要现象 (1) 以前的记忆机制只保存原始对话文本 (Q-R 对) 作为记忆，这需要对历史进行反复推理。我们的方法将想法保存在内存缓存中，无需反复推理即可直接调用。(2) 以前的记忆机制只支持简单的读写 (插入) 操作，而我们的方法提供了更多的记忆操作方式。(3) 以前的一些内存机制将标记存储在内存中，这就需要为应用调整 LLM 架构 (LLM-aware)。我们的方法被设计为一个与 LLM 无关的模块，可以很容易地与其他 LLM 结合使用。

4 实验

4.1 实验设置

4.1.1 数据集。我们使用了三个数据集来证明所提方法的有效性。

- **KdConv 数据集：**KdConv 是一个中文多领域知识驱动会话基准[35]，将话题建立在知识图谱的基础上。
图，涉及三个领域 (电影、音乐和旅游) 的 4.5K 个对话和 86K 个语句。平均回合数为 19。
- **生成虚拟数据集 (GVD)：**生成的虚拟数据集 (GVD) 是一个长期的聊天数据集 [32]，涉及 15 个虚拟用户 (ChatGPT)，历时 10 天。
10 天。对话是使用预先定义的主题合成的，包括英语和汉语。测试集 [32] 人工构建了 194 个查询问题 (97 个英文问题和 97 个中文问题)，以评估 LLM 是否能准确回忆起记忆并生成适当的答案。
- **真实世界医疗数据集 (RMD)：**为了评估所提出的记忆机制在**真实世界医疗数据集 (RMD)** 中的有效性，我们使用了 **"真实世界医疗数据集"**。
为了评估所提出的记忆机制在现实世界中的有效性
为了评估所提出的记忆机制在真实世界场景中的有效性，我们手动收集并构建了一个数据集，其中包含 1,800 个医疗保健消费者的对话。在测试集中，我们使用了 80 个对话来评估 LLM 是否能提供准确的诊断。

4.1.2 LLM。我们整合了两个功能强大的 LLM，以证明所提出的 TiM 机制的有效性。这些 LLM 原本缺乏长期记忆和对长期对话的特定适应性。这些 LLM 的详细介绍如下。

- **ChatGLM [17]：**ChatGLM 是基于通用语言模型 (GLM) 框架的开源双语语言模型[17]。
[17].该模型包含 62 亿个具有特定操作定时功能的参数，涉及监督微调、反馈引导和人工反馈强化学习。
- **百川2[36]：**百川 2 是一个开源的大规模多语言语言模型，包含 130 亿个参数。
该模型是在 2.6 万亿个词库上从头开始训练的。该模型擅长对话和语境理解。

4.1.3 基线。一种基线是在不使用任何内存机制的情况下回答问题。另一种基线是 SiliconFriend [32]，它是一种经典的存储机制。

它可以将原始文本存储到内存中，并支持读取操作。

4.1.4 评估协议。根据 [32]，我们采用了三个指标来评估所提出方法的性能。

- **检索准确性**评估相关记忆是否被成功调用 (标签：{0: 否; 1: 是})。
- **响应正确性**评估是否正确回答了探测问题 (标签：{0: 错误; 0.5: 部分; 1: 正确})。
- **上下文连贯性**评估回答是否自然连贯，*例如*，是否将对话上下文与检索记忆联系起来 (标签：{0: 错误; 0.5: 部分; 1: 正确})。
上下文连贯度：{0: 不连贯; 0.5: 部分连贯; 1: 连贯}。

为了公平起见，在评估过程中，所有 LLM 的预测结果首先会被洗牌，确保人类评估者不知道结果来自哪个 LLM。然后由人工评估得出最终评估结果。

4.2 比较结果

4.2.1 GVD 数据集上的结果。我们在 GVD 数据集的英文和中文测试集上评估了我们的方法。从表 2：中可以看出以下几点 (1) 与 SiliconFriend [32] 相比，我们的方法在所有指标上都表现优异，尤其是在上下文一致性方面，这表明 TiM 机制非常有效。(2) TiM 在两种语言上都取得了更好的结果。中文的性能提升幅度大于英文，这可能与 LLM 的能力有关。

4.2.2 KdConv 数据集上的结果。表 2 展示了 KdConv 数据集上的比较结果。我们针对不同的主题 (电影、音乐和旅游) 对两种不同的 LLM 和 TiM 进行了评估。如表 2 所示，在所有主题中，我们的方法都能获得最佳结果。我们的方法可以达到很高的检索准确率，从而回忆起相关的想法。在没有记忆机制的情况下，由于缺乏长期记忆能力，这些 LLM 通常会表现出较低的回答正确率，而 TiM 则能很好地消除这一负面问题。此外，TiM 还有助于提高回答的上下文连贯性。

4.2.3 RMD 数据集的结果表 2 报告了 RMD 数据集的比较结果，该数据集包含医生和患者之间的真实对话。如表 2，所示我们的方法可以提高真实世界医疗对话的整体响应性能。具体来说，使用 TiM 后，ChatGLM 和 Baichuan2 都能提高它们在长期对话中的能力，*并*显著提高了应答正确性和上下文连贯性。其主要原因是 TiM 与人类记忆的工作流程更为相似，可以增强 LLM 的能力，使其做出更像人类的回答。

4.3 更多分析

4.3.1 检索时间。我们报告了检索时间的比较结果。基线方法是计算问题与整个记忆之间的成对相似度，这也是之前大多数机制的默认检索方式。对于基线我们的方法，内存长度都是 140，内存上下文是固定的。表 2 显示了进行一次检索的时间成本。表 2

表 2：三个数据集的比较结果。从内存缓存中调用前 5 个想法。

数据集	LLM	语言/主题	内存	检索准确性	反应正确性	上下文一致性
GVD	聊天GLM	英语/开放	硅友	0.809	0.438	0.680
			TiM (我们的)	0.820	0.450	0.735
		中文/开放	硅友	0.840	0.418	0.428
			TiM (我们的)	0.850	0.605	0.665
Kdconv	聊天GLM	中文/电影	)	-	0.657	0.923
			TiM (我们的)	0.920	0.827	0.943
		中文/音乐	)	-	0.666	0.910
			TiM (我们的)	0.970	0.826	0.926
		中文/旅游	)	-	0.735	0.906
			TiM (我们的)	0.940	0.766	0.912
	百川2	中文/电影	)	-	0.360	0.413
			TiM (我们的)	0.913	0.743	0.870
		中文/音乐	)	-	0.253	0.283
			TiM (我们的)	0.933	0.757	0.897
聊天GLM		中文/医学	)	-	0.806	0.893
			TiM (我们的)	0.900	0.843	0.940
	百川2	中文/医学	)	-	0.806	0.893
			TiM (我们的)	0.900	0.843	0.940
		中文/医学	)	-	0.806	0.893
			TiM (我们的)	0.900	0.843	0.940
RMD			)	-	0.506	0.538
			TiM (我们的)	0.873	0.538	0.663

表 3：检索时间比较。基线计算问题与记忆之间的成对相似性。

方法	检索时间 (毫秒)	基线
	0.6287	
我们的 (TiM)	0.5305	

可以看出，与基线方法相比，我们的方法可以减少约 0.1 毫秒的检索时间。

4.3.2 Top-k Recall. 我们报告了在 Kdconv 数据集（旅行）上使用不同k 值时的检索准确率。如图 6，所示top-1 检索准确率高于 0.7。随着k 值的增加，整体检索准确率也在提高，其中前 10 名的检索准确率可达 0.973。此外，如表 2 所示，top-5 召回率可以显著提高现有长期会话 LLM 的性能。

4.4 行业应用

在本节中，我们以 ChatGLM 和 TiM 为基础，开发了一个医疗代理（命名为 TiM-LLM），用于病人与医生之间的对话。

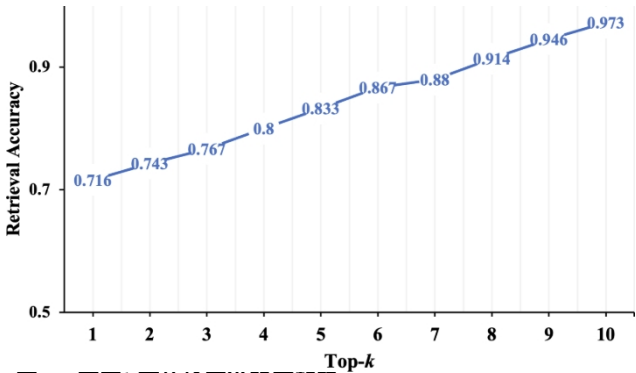


图 6：不同k 下的检索准确率趋势。

如图 7）。所示需要注意的是，TiM-LLM 只是一个辅助工具，供临床医生根据患者的需求提供治疗方案和医疗建议。

图 7 展示了病人和医生之间的真实对话，其中医疗代理在有 TiM 和没有 TiM 的情况

下给出了临床诊断结果。如图 7，所示如果没有 TiM，医疗代理可能很难回忆起以前的症状、

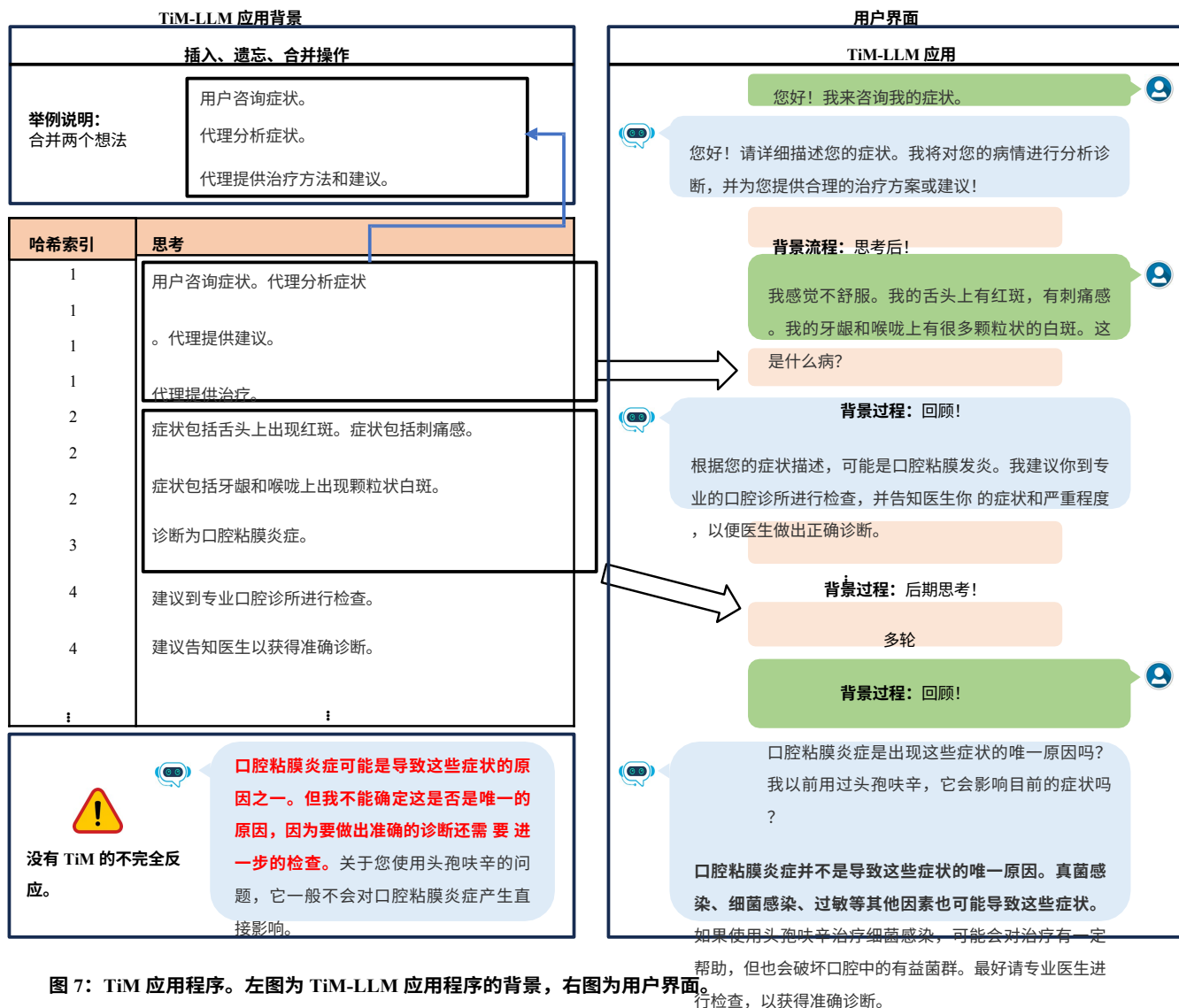


图 7：TiM 应用程序。左图为 TiM-LLM 应用程序的背景，右图为用户界面。

在 TiM-LLM 应用程序的帮助下，医疗人员可以回忆起相关的症状，并对其进行综合评估（红色部分），例如，医疗人员忘记了之前的症状，因此无法确定口腔粘膜炎症是否是唯一的病因。在 TiM 的辅助下，医疗代理可以回忆起相关症状，并对病人的疾病有一个全面的了解。因此，它能提供准确的诊断和治疗（粗体部分）。

5 结论

在这项工作中，我们提出了一种名为 TiM 的新型记忆机制，以解决记忆增强型 LLM 中思想偏差的问题。通过将历史想法存储在进化内存中，TiM 使 LLMs 能够回忆起相关想法并将其纳入对话，而无需反复推敲。TiM 包括两个关键阶段：生成前的回忆和生成后的后思考。此外，TiM 利用几个基本原则来组织记忆中的想法，从而实现记忆的动态更新。此外，我们还在 TiM 中引入了位置敏感

哈希算法（Hashing），以实现对话的高效检索。在真实世界和模拟对话中进行的定性和定量实验证明了将 TiM 装备到 LLM 中的显著优势。总之，TiM 是一种用于提高人与人工智能长期交互的响应质量和一致性的方法。

参考文献

- [1] OpenAI.Chatgpt.2022.
- [2] OpenAI.Gpt-4 技术报告。2023.
- [3] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang.Fingpt: 开源 金融大型语言模型。 *arXiv preprint arXiv:2306.06031*, 2023.
- [4] 张洪波、陈俊英、蒋锋、于飞、陈志宏、李建全、陈桂明、吴湘波、张志毅、肖庆英等:《Huatuogpt, towards taming language model to be a doctor》, *arXiv preprint arXiv:2305.15075*, 2023.
- [5] Tyna Eloundou, Sam Manning, Pamela Mishkin 和 Daniel Rock. Gpts are gpts: *arXiv preprint arXiv:2303.10130*, 2023.
- [6] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa.大语言模型是零射推理器。 *神经 信息处理系统进展*, 35:22199-22213, 2022 年。
- [7] Qi Liu, Dani Yogatama, and Phil Blunsom.关系记忆增强语言模型。 *计算语言学协会论文集*, 10:555-572, 2022.
- [8] Quentin Fournier, Gaétan Marceau Caron 和 Daniel Aloise. 关于更快更轻变压器的实用调查。 *ACM Computing Surveys*, 55 (14s) : 1-40, 2023.
- [9] Jason Phang, Yao Zhao, and Peter J Liu.为长输入摘要有效扩展变换器的研究》, *arXiv preprint arXiv:2208.04347*, 2022.
- [10] 王伟志、董莉、程浩、刘晓东、闫西峰、高剑锋和魏福如。用长期记忆增强语言模型。 *arXiv preprint arXiv:2306.07174*, 2023.
- [11] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark 等: 通过从数万亿词条中检索改进语言模型。 *国际机器学习大会*, 第 2206- 2240 页。PMLR, 2022.
- [12] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova.伯特: 用于语言理解的深度双向变换器的预训练。 In *Proceedings of NAACL-HLT*, volume 1, page 2, 2019.
- [13] Daniel Adiyardana, Minh-Thang Luong, David R So, Jamie Hall, Noah Fiedel, Romal Thoppilan, Zi Yang, Apoorv Kulshreshtha, Gaurav Nemade, Yifeng Lu, et al. Towards a human-like open-domain chatbot. *arXiv preprint arXiv:2001.09977*, 2020.
- [14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou.自洽性改善语言模型中的思维推理链。 *arXiv预印本 arXiv:2203.11171*, 2022。
- [15] 约翰·邓洛斯基和珍妮特·梅特卡夫。 *元认知*。赛奇出版社, 2008 年。
- [16] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya Razenshteyn, and Ludwig Schmidt.角距离的实用和最优 lsh。 *神经 信息处理系统进展*, 2015年28期。
- [17] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. Glm-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414*, 2022.
- [18] Biao Zhang, Barry Haddow, and Alexandra Birch.提示机器翻译的大型语言模型: *arXiv preprint arXiv:2301.07069*, 2023.
- [19] Boyu Zhang, Hongyang Yang, Tianyu Zhou, Ali Babar, and Xiao-Yang Liu.通过检索增强的大语言 模型加强金融情感分析。 *arXiv 预印本 arXiv:2310.04027*, 2023.
- [20] Jiaxian Guo, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Boyang Li, Dacheng Tao and Steven Hoi.从图像到文字提示: 使用冷冻大语言模型的零镜头视觉问题解答。 *IEEE/CVF 计算机视觉与模式识别会议论文集*, 第 10867- 10877 页, 2023 年。
- [21] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: 用路径扩展语言建模。 *arXiv preprint arXiv:2204.02311*, 2022.
- [22] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [23] Rohan Taori, Ishana Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang 和 Tatsunori B Hashimoto.斯坦福羊驼: 指令跟随骆驼模型, 2023。
- [24] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz 和 Dan Roth. 通过大型预训练语言模型进行自然语言处理的最新进展: A survey. *ACM Computing Surveys*, 56 (2) : 1-40, 2023.
- [25] Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski 和 Rosanne Liu. 即插即用的语言模型: 可控文本生成的简单方法。 *arXiv preprint arXiv:1912.02164*, 2019.
- [26] Tomasz Korbak, Kejian Shi, Angelica Chen, Rasika Vinayak Bhalerao, Christopher Buckley, Jason Phang, Samuel R Bowman, and Ethan Perez.用人类偏好预训练语言模型。 *国际机器学习大会*, 第 17506-17533 页。PMLR, 2023。
- [27] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang 和 Weizhu Chen. Lora: Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [28] 孟莲、黄敏莉。利用长短期记忆网络进行对话意图分类。 *自然语言处理与中文计算》: 第六届 CCF 国际会议, NLPCC 2017, 中国大连, 2017 年 11 月 8-12 日, Proceedings 6*, 第 42-50 页。Springer, 2018.
- [29] Alex Graves, Greg Wayne, and Ivo Danihelka. 神经图灵机。 *arXiv preprint arXiv:1410.5401*, 2014.
- [30] Jing Xu, Arthur Szlam, and Jason Weston.超越金鱼记忆: *arXiv preprint arXiv:2107.07567*, 2021.
- [31] 徐新潮、苟志斌、吴文权、牛正宇、吴华、王海峰和王世航。好久不见! 具有长期 角色记忆的开放域对话。 *arXiv 预印本 arXiv:2203.05797*, 2022。
- [32] 钟万钧、郭良红、高琦琦、王彦林。记忆库: 用长期记忆增强大型语言模型。 *arXiv preprint arXiv:2305.10250*, 2023.
- [33] Kinnian Liang, Bing Wang, Hui Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhoujun Li.利用自控内存系统释放大规模语言模型的无限长输入容量。 *ArXiv 预印本 arXiv:2304.13343*, 2023.
- [34] Gabor Angeli, Melvin Jose Johnson Premkumar 和 Christopher D Manning.利用语言结构进行开放域信息提取。 In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 344-354, 2015.
- [35] Hao Zhou, Chuji Zheng, Kaili Huang, Minlie Huang, and Xiaoyan Zhu.KdConv: 面向多轮知识驱动的中文多领域对话数据集 对话。 *ACL*, 2020.
- [36] 杨爰元、肖斌、王冰宁、张伯荣、尹超、吕晨旭、潘达、王滇、闫东、杨帆等:《百川2: 开放的大规模语言模型》。 *arXiv 预印本 arXiv:2309.10305*, 2023。